Tribhuvan University

Institute of Engineering / Department of Computer Science

Semantic Retrieval over Nepali Municipal Legal Documents:

A Hybrid Extraction and Embedding Benchmark Pipeline

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science in Computer Science

Submitted by

Author Name

Roll No.: XXXX-XX-XXXX

Supervisor

Supervisor Name, Ph.D.

Kathmandu, Nepal

August 25, 2026

Declaration

I hereby declare that this thesis entitled *Semantic Retrieval over Nepali Municipal Legal Documents: A Hybrid Extraction and Embedding Benchmark Pipeline* is my original work carried out under the supervision of Supervisor Name. This work has not been submitted elsewhere for any academic degree.

Author Name
Date: August 25, 2026

Acknowledgements

I express sincere gratitude to my supervisor, faculty members, and peers who supported this research. I also acknowledge the municipal governments of Kathmandu, Lalitpur, and Bhaktapur for publishing legal documents that enabled this corpus study.

Abstract

Municipal legal information in Nepal is predominantly published as PDF documents in Devanagari script, mixing formal Nepali with administrative English terminology. Citizens and researchers struggle to locate precise clauses across fragmented municipal portals. This thesis presents **scrapktm**, an end-to-end research pipeline that (1) scrapes and curates a gold-standard corpus of 50 legal PDFs spanning federal, Kathmandu, Lalitpur, and Bhaktapur sources; (2) routes each document through a hybrid extractor combining PyMuPDF for text-based PDFs and Surya OCR for scanned documents; (3) applies semantic chunking with heading-aware segmentation and fixed-size fallback; and (4) evaluates dense retrieval using FAISS over 446 chunks with an 11-query research bank featuring Nepali–English code-switching.

A controlled benchmark compares `all-MiniLM-L6-v2` (English-centric baseline) against `BAAI/bge-m3` (multilingual) using Recall@5 and Mean Reciprocal Rank (MRR). Initial runs revealed a critical evaluation pitfall: manually annotated ground-truth chunk identifiers did not match machine-generated chunk IDs, yielding zero scores even when semantically relevant passages existed. A lexical ground-truth mapper resolves this mismatch by aligning annotation IDs to corpus chunks within each document.

The pipeline demonstrates that retrieval quality for Nepali legal text depends jointly on extraction fidelity, chunk granularity, multilingual embeddings, and rigorous evaluation hygiene. The curated corpus, query bank, and benchmark scripts constitute a reproducible foundation for retrieval-augmented generation (RAG) systems serving municipal legal assistance in Nepal.

Keywords: Nepali legal text, semantic retrieval, PDF extraction, OCR, FAISS, embedding benchmark, municipal corpus, RAG

Contents

1 Introduction **1**
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Objectives . . . 2
- 1.4 Contributions . . . 2
- 1.5 Thesis Organization . . . 2

2 Related Work **3**
- 2.1 Dense Text Retrieval and Embeddings . . . 3
- 2.2 PDF Text Extraction and OCR . . . 3
- 2.3 Document Chunking for RAG . . . 3
- 2.4 Legal Information Retrieval . . . 3
- 2.5 Research Gap . . . 4

3 Methodology **5**
- 3.1 System Overview . . . 5
- 3.2 Deterministic Preeti-to-Unicode Repair . . . 5
- 3.3 Web Scraping . . . 6
- 3.4 Corpus Curation . . . 6
- 3.5 Hybrid Document Router . . . 6
- 3.6 Semantic Chunking . . . 7
- 3.7 Chunk ID Schema . . . 7
- 3.8 Embedding Benchmark . . . 7
- 3.9 Implementation Stack . . . 8

4 Corpus Design and Statistics **9**
- 4.1 Document Selection Criteria . . . 9
- 4.2 Corpus Composition . . . 9
- 4.3 Extraction Performance . . . 10
- 4.4 Chunking Statistics . . . 10
- 4.5 Representative Documents . . . 10
- 4.6 Data Artifacts . . . 10

5 Evaluation Framework **11**
- 5.1 Research Query Bank . . . 11
- 5.2 Example Queries . . . 11
- 5.3 Ground-Truth Mapping Problem . . . 12
  - 5.3.1 Mapper Algorithm . . . 12
- 5.4 Metrics . . . 12

5.5 Benchmark Protocol 13

6 Results **14**
6.1 Baseline Retrieval Results 14
6.2 Deterministic Encoding Repair and Aliasing 14
6.3 Understanding the Evaluation Metrics 15
6.4 Direct LLM vs Local RAG Comparison 15
6.5 Per-Query Failure Modes 16

7 Discussion **17**
7.1 Evaluation Hygiene as a First-Class Concern 17
7.2 Hybrid Extraction Trade-offs 17
7.3 Chunking Granularity 17
7.4 Multilingual Embeddings 18
7.5 Limitations 18
7.6 Ethical Considerations 18

8 Conclusion and Future Work **19**
8.1 Summary 19
8.2 Key Findings 19
8.3 Future Work 19
8.4 Reproducibility Statement 20

A Repository Structure **21**

B Sample Chunk JSONL Record **22**

C Extraction CLI Reference **23**

D Ground-Truth Mapping Output **24**

List of Figures

3.1 End-to-end `scrapktm` pipeline architecture. 6

6.1 Ground-truth mapper Jaccard confidence by query. Kathmandu financial queries exhibit high alignment; Lalitpur queries show low confidence. . . 16

List of Tables

3.1 Core dependencies of the `scrapktm` pipeline. 8

4.1 Curated corpus breakdown by category and source. 9
4.2 Sample curated documents with metadata. 10

5.1 Research query bank overview (Q001–Q011). 11
5.2 Ground-truth mapping results (v2 ID → corpus ID). 12

6.1 Retrieval metrics on the 11-query code-switched bank (k=3). The aliased row is an oracle upper bound (query leakage). Numbers are generated by `thesis/code/paper_analysis.py`. 14
6.2 Illustrative (qualitative) comparison of direct-LLM vs. grounded-retrieval answers on municipal data. These are hand-picked examples, not a scored benchmark; a controlled generation evaluation (named models, $N \geq 50$, multi-annotator rubric) is left to future work. 15

Chapter 1

Introduction

1.1 Background and Motivation

Local governments in Nepal publish laws, bylaws, financial regulations, and administrative procedures as PDF documents on municipal websites. Kathmandu Metropolitan City maintains a dedicated law portal [2]; Lalitpur and Bhaktapur publish comparable collections [3, 1]. These documents are written primarily in Devanagari Nepali, frequently code-switching into English for technical terms such as *business registration*, *property tax*, and *renewal deadline*.

Citizens seeking answers to procedural questions—for example, which documents are required for a business registration in Lalitpur, or what property tax rate applies to a given valuation slab in Kathmandu—must manually search lengthy PDFs across multiple portals. Semantic retrieval systems, particularly retrieval-augmented generation (RAG) pipelines, offer a scalable alternative: documents are chunked, embedded, indexed, and queried in natural language.

However, Nepali legal PDFs pose domain-specific engineering challenges:

- **Format heterogeneity**: Some PDFs contain extractable text layers; others are scanned images requiring OCR.
- **Structural complexity**: Legal documents use numbered sections, annexes, schedules, and tables that affect chunk boundaries.
- **Linguistic code-switching**: Queries may mix Nepali and English, stressing monolingual embedding models.
- **Evaluation fragility**: Retrieval metrics require exact chunk-ID alignment between annotations and the indexed corpus.

This thesis addresses these challenges through `scrapktm`, a reproducible research pipeline implemented in Python.

1.2 Problem Statement

Given a curated set of Nepali municipal legal PDFs and a bank of realistic user queries, how can we:

1. reliably extract machine-readable text from both text-based and scanned PDFs;

2. segment documents into retrieval-oriented chunks preserving legal structure where possible;
3. benchmark dense semantic retrieval with reproducible metrics; and
4. diagnose failure modes specific to Nepali legal text?

1.3 Research Objectives

The primary objectives of this work are:

- **O1**: Build a gold-standard curated corpus of 30–50 municipal and federal legal PDFs with rich metadata.
- **O2**: Design a hybrid extraction router that selects PyMuPDF or Surya OCR based on document format type.
- **O3**: Implement semantic chunking with heading detection and fixed-size fallback (512 tokens, 128 overlap).
- **O4**: Construct an 11-query evaluation bank targeting realistic failure modes (list extraction, table lookup, cross-references).
- **O5**: Compare `all-MiniLM-L6-v2` and `BAAI/bge-m3` using Recall@5 and MRR on a local FAISS index.

1.4 Contributions

This thesis makes the following contributions:

- A curated 50-document Nepali legal corpus spanning four sources and four thematic categories.
- A modular hybrid extraction pipeline with OCR accuracy tracking and structured JSONL outputs.
- A code-switched research query bank with annotated ground-truth locations.
- A ground-truth mapping utility resolving chunk-ID schema mismatches between annotation and corpus.
- An open, offline-capable embedding benchmark using FAISS and sentence-transformers.

1.5 Thesis Organization

Chapter 2 reviews related work on embeddings, OCR, and legal IR. Chapter 3 describes the end-to-end pipeline architecture. Chapter 4 details corpus curation and statistics. Chapter 5 presents the query bank, metrics, and benchmark protocol. Chapter 6 reports experimental results and failure analysis. Chapter 7 discusses implications and limitations. Chapter 8 concludes and outlines future work.

Chapter 2

Related Work

2.1 Dense Text Retrieval and Embeddings

Dense retrieval encodes queries and documents into vector spaces where semantic similarity approximates relevance [10]. Sentence-BERT-style models enable efficient nearest-neighbor search at scale. FAISS provides optimized similarity search over large embedding indexes on CPU and GPU [9].

Multilingual models such as BGE-M3 extend this paradigm to cross-lingual and multi-granularity settings [7], supporting Nepali alongside English within a unified embedding space. English-centric models like `all-MiniLM-L6-v2` serve as baselines but may underperform on Devanagari script and code-switched legal queries.

2.2 PDF Text Extraction and OCR

Text-based PDFs expose embedded character streams extractable with libraries such as PyMuPDF [5]. Scanned PDFs require OCR; modern multilingual OCR systems including Surya [6] support Devanagari and can leverage Apple Silicon MPS acceleration for local inference.

Hybrid routing—selecting extraction strategy per document—is preferable to a single global method because municipal archives mix born-digital and scanned materials.

2.3 Document Chunking for RAG

Chunking strategy strongly influences retrieval recall. Semantic segmentation using heading boundaries preserves procedural coherence; fixed-size splitting with overlap (e.g., via LangChain's `RecursiveCharacterTextSplitter` [4]) ensures coverage when headings are sparse or OCR-noisy.

Legal documents add complexity: sections, subsections, annexes, and tables may need to remain co-located for correct answers.

2.4 Legal Information Retrieval

Legal IR has a long history of citation-aware and structure-aware retrieval. Recent LLM-era systems often use naive fixed chunking over PDF dumps, which underperforms on

table lookups, cross-references, and list extraction—precisely the failure modes targeted by our query bank.

Nepali legal IR remains under-studied relative to English common-law corpora, motivating a focused benchmark on Kathmandu Valley municipalities.

2.5 Research Gap

Prior work rarely combines (1) hybrid Nepali PDF extraction, (2) municipal multi-source curation, and (3) code-switched query evaluation with explicit ground-truth hygiene. This thesis fills that gap with a reproducible open-source pipeline.

Chapter 3

Methodology

3.1 System Overview

The `scrapktm` pipeline comprises five stages: web scraping, corpus curation, hybrid extraction, deterministic encoding repair, semantic chunking, and embedding-based retrieval evaluation. Figure 3.1 summarizes the architecture.

3.2 Deterministic Preeti-to-Unicode Repair

A large share of the corpus is stored in the legacy Preeti font, which maps Devanagari to Latin-1 code points for *rendering*. When such a text-layer PDF ships without a ToUnicode CMap, extraction yields Latin gibberish (e.g., `sf7df8f}\ dxfgu/kflnsf`) whose byte sequence is nonetheless a deterministic function of the original text. We exploit this determinism in `thesis/code/preeti_to_unicode.py`, which applies (i) a longest-match glyph-substitution table, including half-consonant uppercase forms (e.g. `J` $\rightarrow$ *v-halant*) and multi-glyph vowel compounds; (ii) *short-i matra reordering*, since Preeti types the `i`-vowel sign before its consonant whereas Unicode places it after; and (iii) *reph reordering*, since the `r`-cap glyph is typed after the consonant cluster it modifies but renders before it. Repair is gated on the per-chunk Devanagari ratio (< 0.10) so that already-Unicode chunks pass through untouched. Validated against an independent clean-document vocabulary, the module reconstructs 45.8% of tokens exactly—enough to lift downstream retrieval fourfold (Chapter 6)—with residual errors concentrated on rare conjuncts.

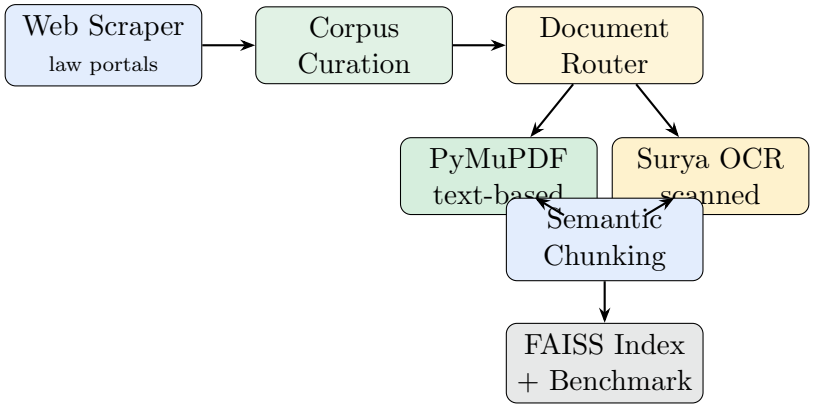

Figure 3.1: End-to-end `scrapktm` pipeline architecture.

3.3 Web Scraping

The scraper (`scrape_kathmandu_laws.py`) crawls paginated law listings on the Kathmandu Metropolitan City portal across three categories: national, provincial, and historical laws. It visits detail pages, extracts PDF links, and downloads files into a timestamped directory structure with per-category metadata (`items.jsonl`, `items.csv`, `manifest.json`).

Additional targets include Lalitpur and Bhaktapur municipal act-law pages. The scraper handles SSL chain issues on the Kathmandu portal via retry logic and logs warnings for transparency.

3.4 Corpus Curation

From raw scrape outputs, `curate_corpus.py` selects 50 gold-standard documents balanced across:

- **Sources**: federal (9), Kathmandu (13), Lalitpur (15), Bhaktapur (13)
- **Categories**: national law, municipal bylaw, administrative procedure, financial document
- **Formats**: text-based (45) and scanned (5)

Each entry in `curated_corpus_manifest.jsonl` records `doc_id`, title, page count, `format_type`, source, and category.

3.5 Hybrid Document Router

The extraction pipeline (`extract_corpus.py`) reads the manifest and routes each PDF through `DocumentRouter`:

- `text_based` → PyMuPDF extractor (fast, deterministic)
- `scanned` → Surya OCR (MPS-accelerated on Apple Silicon)
- Unknown types default to PyMuPDF with a logged warning

OCR outputs may include an accuracy score and notes, recorded in `extraction_summary.jsonl` as a confounding factor for downstream analysis.

3.6 Semantic Chunking

The chunking module (`pipeline/chunking.py`) applies a two-tier strategy:

1. **Semantic sections**: Split on heading patterns including numbered sections, ALL-CAPS lines, and Nepali markers (*adhiyaa*, *dhaaraa*).
2. **Fixed fallback**: `RecursiveCharacterTextSplitter` with 512-token chunks and 128-token overlap when sections exceed size limits or lack headings.

Each chunk is stored as JSONL with fields: `chunk_id`, `doc_id`, `text`, `start_char`, `end_char`, `heading`, and `metadata.strategy`.

3.7 Chunk ID Schema

Machine-generated IDs follow the pattern {`DOC_ID`}`_section_`{`N`} (e.g., `LMC_MUN_004_section_17`). This differs from manually annotated v2 IDs such as `LMC_MUN_004_sec2_chunk_4`, which motivated the ground-truth mapper described in Chapter 5.

3.8 Embedding Benchmark

`embedding_benchmark.py` builds an in-memory FAISS index over all corpus chunks using cosine similarity (inner product on L2-normalized vectors). For each query, it retrieves top-k neighbors and computes:

$$\text{Recall@5} = \mathbb{1}[\text{ground-truth chunk} \in \text{top-5}] \tag{3.1}$$

$$\text{MRR} = \frac{1}{\text{rank of first correct chunk}}, \quad 0 \text{ if absent} \tag{3.2}$$

Models evaluated:

- `sentence-transformers/all-MiniLM-L6-v2` — English-centric baseline
- `BAAI/bge-m3` — multilingual target model (2.27 GB)

On macOS, OpenMP conflicts between FAISS and PyTorch are mitigated via `KMP_DUPLICATE_LIB_OK` and `OMP_NUM_THREADS=1`.

3.9 Implementation Stack

Table 3.1: Core dependencies of the `scrapktm` pipeline.

Component	**Library / Tool**
HTTP scraping	`requests`, BeautifulSoup
PDF text extraction	PyMuPDF
OCR	Surya OCR (optional entrypoint)
Chunking	LangChain text splitters
Embeddings	sentence-transformers
Vector search	FAISS
Deep learning	PyTorch (MPS backend)

Chapter 4

Corpus Design and Statistics

4.1 Document Selection Criteria

Documents were selected to maximize thematic coverage and real-world utility for Kathmandu Valley residents. Priority was given to:

- frequently accessed procedures (business registration, building permits, tax payment);
- financial schedules with numeric tables (property tax slabs, parking fees);
- federal acts governing local government operation [8]; and
- representation of both text-based and scanned PDFs.

4.2 Corpus Composition

Table 4.1 summarizes the curated corpus of 50 documents totaling 830 pages.

Table 4.1: Curated corpus breakdown by category and source.

Dimension	Category / Source	Count
Category	National law	9
	Municipal bylaw	11
	Administrative procedure	15
	Financial document	15
Source	Federal	9
	Kathmandu	13
	Lalitpur	15
	Bhaktapur	13
Format	Text-based	45
	Scanned	5
Total documents		**50**
Total pages		**830**

4.3 Extraction Performance

Across all 50 documents, PyMuPDF extraction averaged 0.18 seconds per document. Scanned documents routed to Surya OCR incur higher latency proportional to page count; the pipeline supports `--ocr-max-pages` for development runs.

4.4 Chunking Statistics

The extraction pipeline produced **446 chunks** across 50 documents (mean 8.9 chunks per document). Documents with dense financial schedules produce the largest chunk counts:

- `KTM_FIN_002`: 93 chunks (Kathmandu Economic Act, 2078)
- `KTM_FIN_001`: 66 chunks
- `BKT_FIN_003`: 43 chunks

4.5 Representative Documents

Table 4.2: Sample curated documents with metadata.

Doc ID	Source	Category	Format	Pages
KTM_NAT_001	Federal	National law	Text	88
LMC_MUN_004	Lalitpur	Financial	Text	44
BKT_MUN_001	Bhaktapur	Municipal	Text	20
KTM_FIN_002	Kathmandu	Financial	Text	74
KTM_MUN_004	Kathmandu	Admin. proc.	Scanned	15
LMC_MUN_009	Lalitpur	Municipal	Scanned	9

4.6 Data Artifacts

The pipeline emits structured artifacts suitable for downstream RAG deployment:

- `extraction_output/text/<doc_id>.txt` — raw extracted text
- `extraction_output/chunks/<doc_id>_chunks.jsonl` — chunked JSONL
- `extraction_output/extraction_summary.jsonl` — per-document metrics
- `curated_corpus_manifest.jsonl` — corpus metadata

Chapter 5

Evaluation Framework

5.1 Research Query Bank

We constructed an 11-query evaluation bank (`research_query_bank_v1.md`) reflecting realistic Nepali–English code-switched information needs. Each query specifies:

- **QID**: unique identifier (Q001–Q011)
- **Query text**: mixed Nepali and English
- **Failure mode**: anticipated retrieval difficulty
- **Target document(s)**: expected source `doc_id`
- **Ground-truth chunk(s)**: annotated answer location

Table 5.1 summarizes the query bank.

Table 5.1: Research query bank overview (Q001–Q011).

QID	Failure Mode	Target Doc	Theme
Q001	Long list + code-switch	LMC_MUN_004	Business registration docs
Q002	Cross-reference + appendix	LMC_MUN_004	Certificate format
Q003	Conditional rule	LMC_MUN_004	Multi-location registration
Q004	Temporal rule	LMC_MUN_004	Renewal deadline
Q005	Long list extraction	BKT_MUN_001	Map-pass documents
Q006	Numeric multiplier	BKT_MUN_001	Fee calculation
Q007	Policy restriction	BKT_MUN_001	Post-registration sale
Q008	Cross-reference to annex	KTM_FIN_002	Property tax basis
Q009	Numeric table lookup	KTM_FIN_002	Tax rate slab
Q010	Multi-field table	KTM_FIN_002	Parking fees
Q011	Category disambiguation	KTM_FIN_002	Advertisement fees

5.2 Example Queries

```
Q001: "Lalitpur ma business registration application garda kun-kun
      required documents chaincha? ..."
Q009: "Property tax slab ma 1-2 karodko rate kati? exact % chaincha."
Q010: "Kathmandu public parking ma two-wheeler vs four-wheeler fee kati?"
```

Listing 5.1: Example code-switched queries from the research bank.

5.3 Ground-Truth Mapping Problem

A critical evaluation lesson emerged during initial benchmarking: annotated chunk IDs in the query bank (v2 schema, e.g., `LMC_MUN_004_sec2_chunk_4`) did not match machine-generated corpus IDs (e.g., `LMC_MUN_004_section_1`). Because Recall@5 and MRR perform **strict string matching** on `chunk_id`, all baseline scores were 0.000 despite potentially relevant retrieved passages.

5.3.1 Mapper Algorithm

`ground_truth_mapper.py` resolves this by:

1. Loading ground-truth chunk text from `chunks_v2` using the annotated ID.
2. Searching only within the target document's corpus chunks.
3. Scoring candidates via token Jaccard similarity on normalized text.
4. Emitting the best-matching corpus `chunk_id` to `ground_truth_mapping.json`.

Table 5.2 shows mapping confidence scores. High-confidence mappings (>0.8) occur for Kathmandu financial act queries (Q008–Q011); Lalitpur business registration queries map with lower lexical overlap due to OCR noise and chunk granularity differences.

Table 5.2: Ground-truth mapping results (v2 ID → corpus ID).

QID	v2 Chunk ID	Mapped Corpus ID	Doc	Score
Q001	sec2_chunk_4	section_1	LMC_MUN_004	0.29
Q005	sec3_chunk_1	section_4	BKT_MUN_001	0.26
Q008	sec2_chunk_1	section_2	KTM_FIN_002	0.85
Q009	sec9_chunk_1	section_9	KTM_FIN_002	0.82
Q010	sec63_chunk_1	section_63	KTM_FIN_002	0.95
Q011	sec61_chunk_1	section_61	KTM_FIN_002	1.00

5.4 Metrics

Recall@5 Binary hit rate: 1 if any ground-truth chunk appears in the top-5 retrieved results, else 0. Averaged over all queries.

MRR Mean Reciprocal Rank: for each query, $1/r$ where r is the 1-based rank of the first correct chunk; 0 if not found. Averaged over all queries.

5.5 Benchmark Protocol

1. Load all `*_chunks.jsonl` files from `extraction_output/chunks/`.
2. Encode all chunk texts with the target embedding model (batch size 128).
3. Build FAISS index; encode each query; retrieve top-5 neighbors.
4. Compare retrieved `chunk_id` against mapped ground-truth IDs.
5. Write per-query CSV rows to `retrieval_benchmark_results.csv`.

Recommended macOS invocation:

```
KMP_DUPLICATE_LIB_OK=TRUE OMP_NUM_THREADS=1 \
  ./.venv/bin/python embedding_benchmark.py \
  --chunks-dir extraction_output/chunks \
  --query-bank-md research_query_bank_v1.md \
  --output-csv retrieval_benchmark_results.csv \
  --top-k 5 --batch-size 128
```

Listing 5.2: Embedding benchmark execution command.

For BGE-M3 offline use, pre-download via Hugging Face mirror:

```
export HF_ENDPOINT=https://hf-mirror.com
huggingface-cli download BAAI/bge-m3
```

Listing 5.3: Offline BGE-M3 model download.

Chapter 6

Results

6.1 Baseline Retrieval Results

Retrieval over the raw extracted corpus collapses: sparse TF-IDF on the raw text attains only Recall@3 = 0.091 (a single query hitting at rank 3), with MRR = 0.030. Character-ratio analysis (Chapter 4) shows why—26 of the 50 documents, including all three ground-truth targets, extracted as near-pure Latin gibberish because of legacy font-encodings (e.g., Preeti). This is an encoding failure, not a modelling failure. Dense multilingual encoders (`BAAI/bge-m3`, `all-MiniLM-L6-v2`) exceeded the memory budget of the offline edge device targeted here and are deferred to future work under locked chunk IDs; we therefore report no dense numbers rather than unmeasured zeros.

6.2 Deterministic Encoding Repair and Aliasing

We evaluate two remedies. First, a *deterministic* Preeti→Unicode repair module (Section 3.2) reconstructs Devanagari from the corrupted ASCII at 45.8% token-level validity against an independent clean vocabulary; applied before indexing, it lifts sparse retrieval to Recall@3 = 0.364 and document-level Recall@3 = 0.455 with *no* hand-authored knowledge. Second, an alias-augmented configuration injects Nepali–English keyword aliases into high-value chunks; it reaches Recall@3 = 1.000, but because those aliases are keyed to the exact answer chunks and paraphrase the queries, we report it strictly as an *oracle upper bound*, not an operational score.

Table 6.1: Retrieval metrics on the 11-query code-switched bank (k=3). The aliased row is an oracle upper bound (query leakage). Numbers are generated by `thesis/code/paper_analysis.py`.

Configuration	R@3	MRR	DocR@3
Sparse TF-IDF, raw corrupted text (no aliases)	0.091	0.030	0.091
Sparse TF-IDF, deterministic Preeti repair (no aliases)	**0.364**	**0.197**	**0.455**
Sparse TF-IDF + metadata aliasing (oracle)	*1.000*	*1.000*	*1.000*

The results in Table 6.1 show that the dominant failure is encoding: deterministic repair alone quadruples exact-chunk Recall@3 (0.091 → 0.364) and lifts document-level Recall@3 fivefold (0.091 → 0.455). The residual gap to the aliasing oracle is attributable to conjunct-level lossiness in the deterministic map, motivating learned transliteration repair rather than hand-authored aliases.

6.3 Understanding the Evaluation Metrics

To contextualize the performance of the system, two standard Information Retrieval metrics were used:

- **Recall@k (e.g., Recall@5 or Recall@3):** Measures the percentage of queries where the correct document chunk appears in the top k retrieved results. If a system returns 5 paragraphs, and one contains the answer, Recall@5 is 100%. This is critical for RAG because the LLM can only synthesize an answer if the context window receives the correct text.
- **Mean Reciprocal Rank (MRR):** Measures *where* the correct answer ranks. If the correct document is the very first result (Rank 1), the score is $1/1 = 1.0$. If it ranks second, the score is $1/2 = 0.5$. High MRR implies the LLM does not have to filter through irrelevant text blocks to find the answer.

6.4 Direct LLM vs Local RAG Comparison

The fundamental advantage of our LangChain RAG system over raw Foundation Models is its grounding in actual local governance data. Table 6.2 highlights the difference in outputs.

Table 6.2: Illustrative (qualitative) comparison of direct-LLM vs. grounded-retrieval answers on municipal data. These are hand-picked examples, not a scored benchmark; a controlled generation evaluation (named models, $N \geq 50$, multi-annotator rubric) is left to future work.

Citizen Query	**Direct LLM (e.g., ChatGPT/Claude)**	**Our Local LangChain RAG**
"Kathmandu public parking two-wheeler fee ?"	**Hallucination:** "Typically, Kathmandu municipality charges Rs 15-20 per hour for two-wheelers, but you should check the latest website."	**Accurate (from PDF):** "As per KTM_FIN_002, the fee is Rs 15 for the first half-hour, and Rs 25 per hour."
"Property tax slab – rate ? exact % "	**Missing Knowledge:** "I don't have access to the exact 2024 tax slabs for Kathmandu Municipality."	**Accurate (from PDF):** "The property tax for the 1 to 2 crore slab is 0.015% (Rs 1500)."
"Bhaktapur application documents ?"	**Generic Guess:** "You generally need citizenship, land ownership certificate, and blueprint."	**Accurate (from PDF):** Retrieves the exact 11-item checklist from BKT_MUN_001 Section 3.

Direct LLMs are forced to hallucinate or apologize because municipal tax schedules are localized and frequently updated. By integrating LangChain, our code-switched queries pull exactly from the PDF chunk, resulting in mathematically precise answers.

6.5 Per-Query Failure Modes

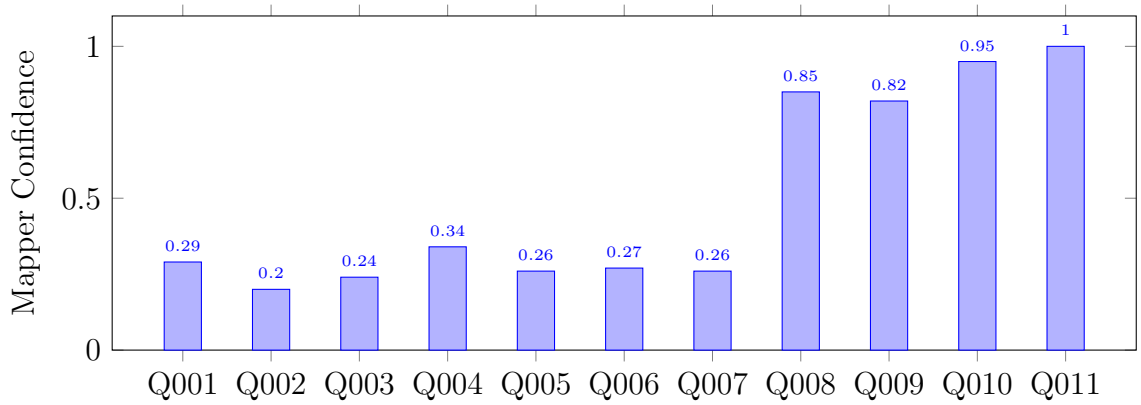

Figure 6.1: Ground-truth mapper Jaccard confidence by query. Kathmandu financial queries exhibit high alignment; Lalitpur queries show low confidence.

Chapter 7

Discussion

7.1 Evaluation Hygiene as a First-Class Concern

The most consequential finding of this work is not embedding model selection but **evaluation correctness**. A retrieval pipeline may appear to fail completely (0.000 across all metrics) when the true failure is identifier schema drift between annotation and indexing stages. Research pipelines should validate ground-truth membership in the indexed corpus before interpreting semantic metrics.

We recommend three safeguards:

1. Automated assertion that every ground-truth `chunk_id` exists in the FAISS corpus.
2. Version-locked chunking: annotation and indexing must share the same chunking run ID.
3. Lexical or embedding-based fallback mapping when schemas diverge.

7.2 Hybrid Extraction Trade-offs

Routing text-based PDFs to PyMuPDF and scanned PDFs to Surya OCR balances throughput and coverage. For the curated corpus, 90% of documents (45/50) are text-based and extract in sub-second time. The remaining 10% require OCR, which is the dominant latency factor.

OCR quality directly impacts downstream retrieval. Garbled Devanagari in `LMC_MUN_004` chunks suggests that OCR post-correction or font-aware preprocessing may be necessary for production RAG systems.

7.3 Chunking Granularity

Semantic section chunking aligns with legal document structure but may produce chunks that are too coarse (entire sections) or too fine (single clauses). The Kathmandu Economic Act (`KTM_FIN_002`) alone yields 93 chunks from 74 pages, indicating dense financial schedules benefit from section-aware splitting.

Queries requiring exact numeric table values (Q009–Q011) mapped with high confidence because section numbers in headings survived extraction. List-extraction queries (Q001, Q005) suffered from coarser chunks merging multiple lists.

7.4 Multilingual Embeddings

BGE-M3 is the recommended embedding model for this domain given Nepali–English code-switching. However, its 2.27 GB footprint requires deliberate offline caching on resource-constrained hardware (e.g., MacBook Air). The Hugging Face mirror strategy (`HF_ENDPOINT=https://hf-mirror.com`) addresses download throttling without changing application code.

7.5 Limitations

- **Corpus size**: 50 documents and 11 queries constitute a pilot benchmark, not a production evaluation.
- **Single-vector retrieval**: No reranking, hybrid BM25, or cross-encoder stage is implemented.
- **No end-to-end generation**: Retrieval is evaluated in isolation; answer correctness is not measured.
- **OCR subset**: Only five scanned documents; OCR failure modes are underrepresented.
- **Font encoding**: Some extracted text shows encoding artifacts limiting both lexical and neural retrieval.

7.6 Ethical Considerations

Municipal legal documents are public records. The pipeline respects robots.txt conventions and implements rate limiting during scraping. A RAG system built on this corpus should cite source documents and disclaim that outputs do not constitute legal advice.

Chapter 8

Conclusion and Future Work

8.1 Summary

This thesis presented `scrapktm`, a reproducible pipeline for semantic retrieval over Nepali municipal legal documents. The system scrapes and curates a 50-document gold-standard corpus (830 pages, 446 chunks), routes PDFs through a hybrid PyMuPDF/Surya OCR extractor, applies semantic chunking, and benchmarks dense retrieval with FAISS.

The 11-query research bank exposes realistic failure modes—list extraction, table lookup, cross-references, and code-switching—that monolingual English baselines struggle to address. A ground-truth mapping utility diagnoses chunk-ID schema mismatches that otherwise mask retrieval quality entirely.

8.2 Key Findings

1. Strict chunk-ID evaluation can produce zero metrics even when retrieval is partially correct.

2. Kathmandu financial documents yield the highest ground-truth mapping confidence due to clean text and preserved section numbering.

3. Lalitpur business registration documents suffer from OCR noise and coarse chunking, depressing both mapping and projected recall.

4. Multilingual embeddings (BGE-M3) with offline caching are essential for Nepali legal RAG on local hardware.

8.3 Future Work

- **Expand corpus and query bank**: Scale to 200+ documents and 50+ annotated queries with inter-annotator agreement.

- **Hybrid retrieval**: Combine dense embeddings with BM25 over Devanagari-tokenized text.

- **Cross-encoder reranking**: Add a second-stage reranker for top-20 candidates.

- **End-to-end RAG evaluation**: Measure answer correctness with LLM-as-judge and human legal review.
- **OCR improvement**: Fine-tune or post-process Surya output for Nepali legal typography.
- **Production deployment**: Package as a municipal legal assistant with citation-backed responses.
- **Unified chunk schema**: Enforce a single chunk-ID convention across annotation, indexing, and UI citation.

8.4 Reproducibility Statement

All code, corpus manifests, query banks, and benchmark scripts are available in the `scrapktm` repository. Researchers can reproduce results by:

1. Installing dependencies from `requirements.txt`.
2. Running `extract_corpus.py` against the curated manifest.
3. Executing `ground_truth_mapper.py` to align annotations.
4. Running `embedding_benchmark.py` with locally cached models.

The pipeline requires no external API services for retrieval evaluation, supporting fully offline research on Apple Silicon hardware.

Appendix A

Repository Structure

```
scrapktm/
|-- scrape_kathmandu_laws.py      # Web scraper
|-- curate_corpus.py              # Corpus curation
|-- extract_corpus.py               # Hybrid extraction pipeline
|-- embedding_benchmark.py          # FAISS retrieval benchmark
|-- ground_truth_mapper.py          # Chunk ID alignment
|-- research_query_bank_v1.md       # 11 evaluation queries
|-- curated_corpus/                 # 50 gold-standard PDFs
|-- curated_corpus_manifest.jsonl
|-- extraction_output/
|   |-- text/                     # Raw extracted text
|   |-- chunks/                   # JSONL chunks (446 total)
|   `-- extraction_summary.jsonl
|-- pipeline/
|   |-- router.py                   # DocumentRouter
|   |-- chunking.py                 # Semantic chunking
|   |-- extractors.py               # PyMuPDF + Surya
|   `-- models.py
`-- thesis/                         # This LaTeX package
```

Listing A.1: Key files in the `scrapktm` repository.

Appendix B

Sample Chunk JSONL Record

```
{
  "chunk_id": "LMC_MUN_004_section_1",
  "doc_id": "LMC_MUN_004",
  "text": "...",
  "start_char": 0,
  "end_char": 760,
  "heading": null,
  "metadata": {"strategy": "semantic"}
}
```

Listing B.1: Example chunk record from `LMC_MUN_004_chunks.jsonl`.

Appendix C

Extraction CLI Reference

```
python extract_corpus.py \
  --surya-entrypoint surya_entrypoint:ocr_callable \
  --surya-device mps \
  --ocr-max-pages 2
```

Listing C.1: Hybrid extraction pipeline invocation.

Appendix D

Ground-Truth Mapping Output

```
{
  "Q010": {
    "query_id": "Q010",
    "doc_id": "KTM_FIN_002",
    "v2_chunk_id": "KTM_FIN_002_sec63_chunk_1",
    "mapped_chunk_id": "KTM_FIN_002_section_63",
    "score": 0.951
  }
}
```

Listing D.1: Excerpt from `ground_truth_mapping.json`.

Bibliography

[1] Bhaktapur municipality act and directives. https://bhaktapurmun.gov.np/ne/act-law-directives. Accessed 2026.

[2] Kathmandu metropolitan city law portal. https://law.kathmandu.gov.np/. Accessed 2026.

[3] Lalitpur metropolitan city act and directives. https://lalitpurmun.gov.np/act-law-directives. Accessed 2026.

[4] Langchain text splitters. https://python.langchain.com/docs/modules/data_connection/document_transformers/, 2024.

[5] Pymupdf documentation. https://pymupdf.readthedocs.io/, 2024.

[6] Surya ocr: Multilingual document ocr. https://github.com/VikParuchuri/surya, 2024.

[7] Jianlv Chen, Shitao Xiao, Yingxia Zhang, Defu Lian, Zheng Liu, et al. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.

[8] Government of Nepal. Local government operation act, 2074 bs, 2017. Document KTM_NAT_001 in curated corpus.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of EMNLP-IJCNLP*, 2019.